```
!pip install opencv-python scikit-learn matplotlib numpy
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

print("Libraries loaded successfully!")
```

```
Libraries loaded successfully!
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

# OpenCV BGR ko RGB mein convert karna
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print("Image loaded successfully!")
print("Image size:", img.shape)
```

```
Image loaded successfully!
Image size: (1536, 1185, 3)
```

```
plt.figure(figsize=(8, 10))

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.show()
```

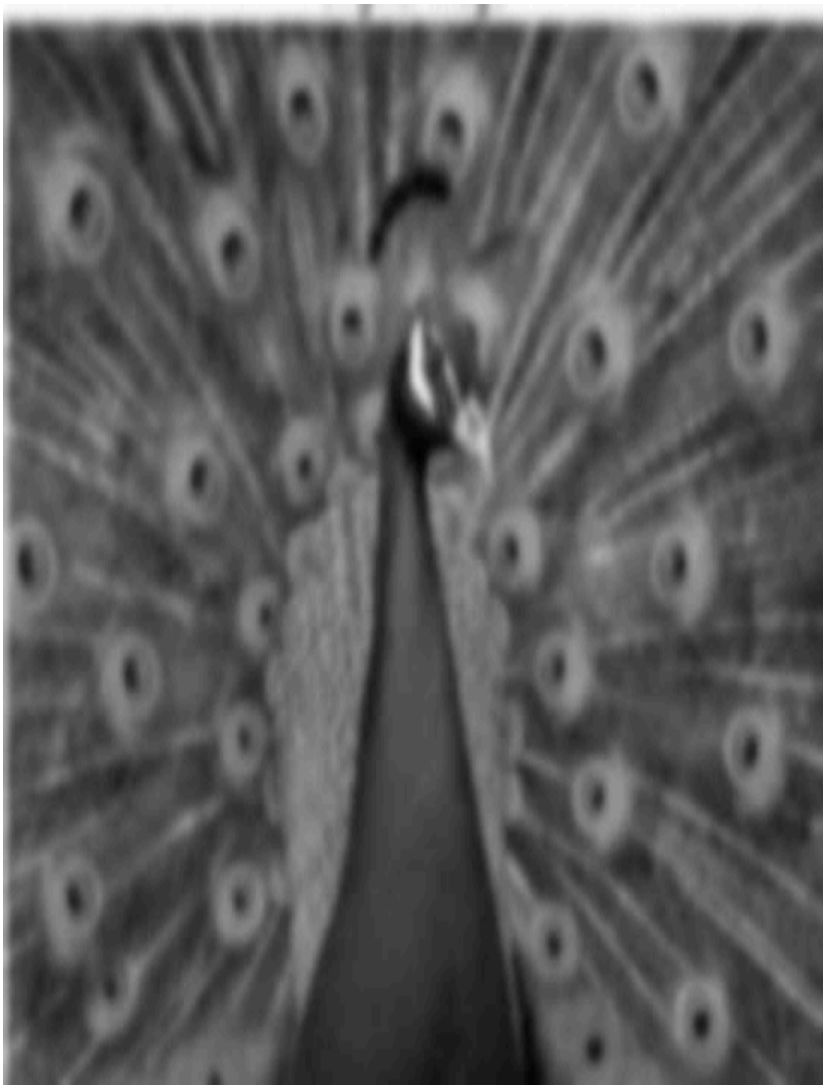


```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 10))

plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.show()
```



```
# Gaussian Blur
blur = cv2.GaussianBlur(gray, (15, 15), 0)

plt.figure(figsize=(14, 7))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



```
ret, global_thresh = cv2.threshold(
    blur,
    127,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(8, 10))

plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.show()

print("Threshold value used:", ret)
```



```
Threshold value used: 127.0
```

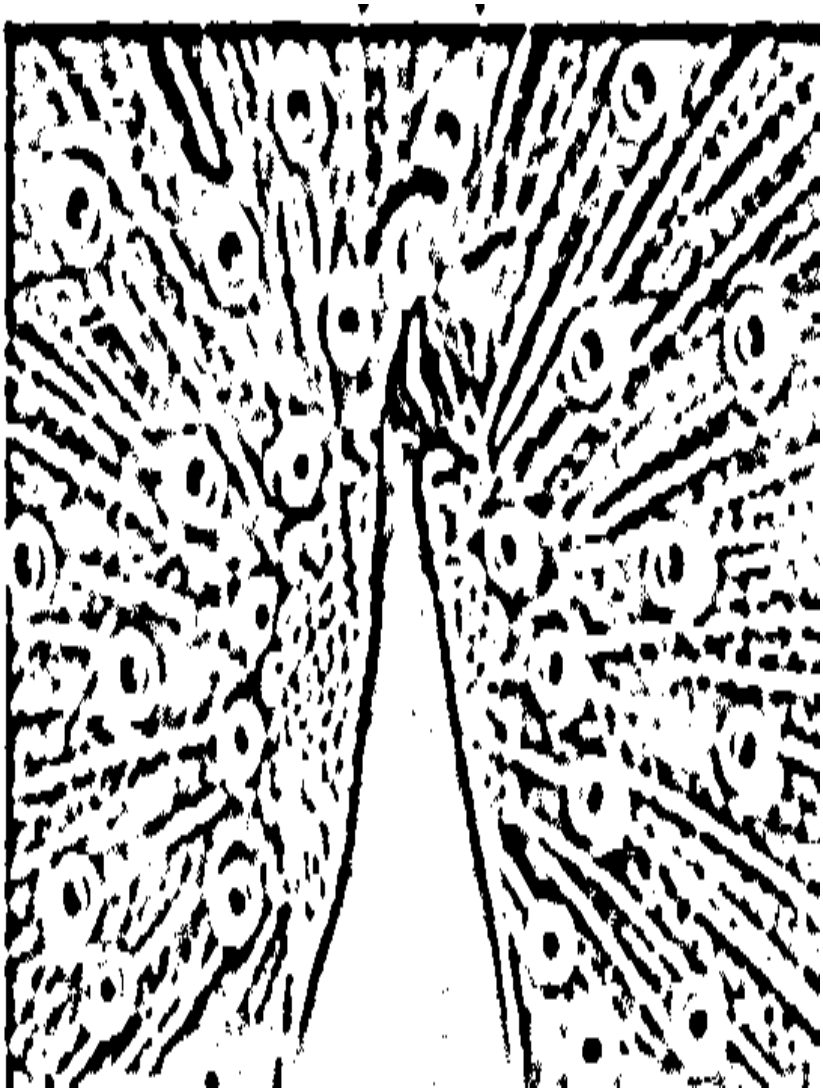
```
otsu_value, otsu_thresh = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

plt.figure(figsize=(8, 10))

plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.show()

print("Automatically selected Otsu threshold:", otsu_value)
```



```
Automatically selected Otsu threshold: 87.0
```

```
adaptive_thresh = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

plt.figure(figsize=(8, 10))

plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.show()
```



```
# Binary image
_, thresh = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Noise removal
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255
```

```
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labelling
num_labels, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1

markers[unknown == 255] = 0

# Apply watershed
watershed_img = img_rgb.copy()

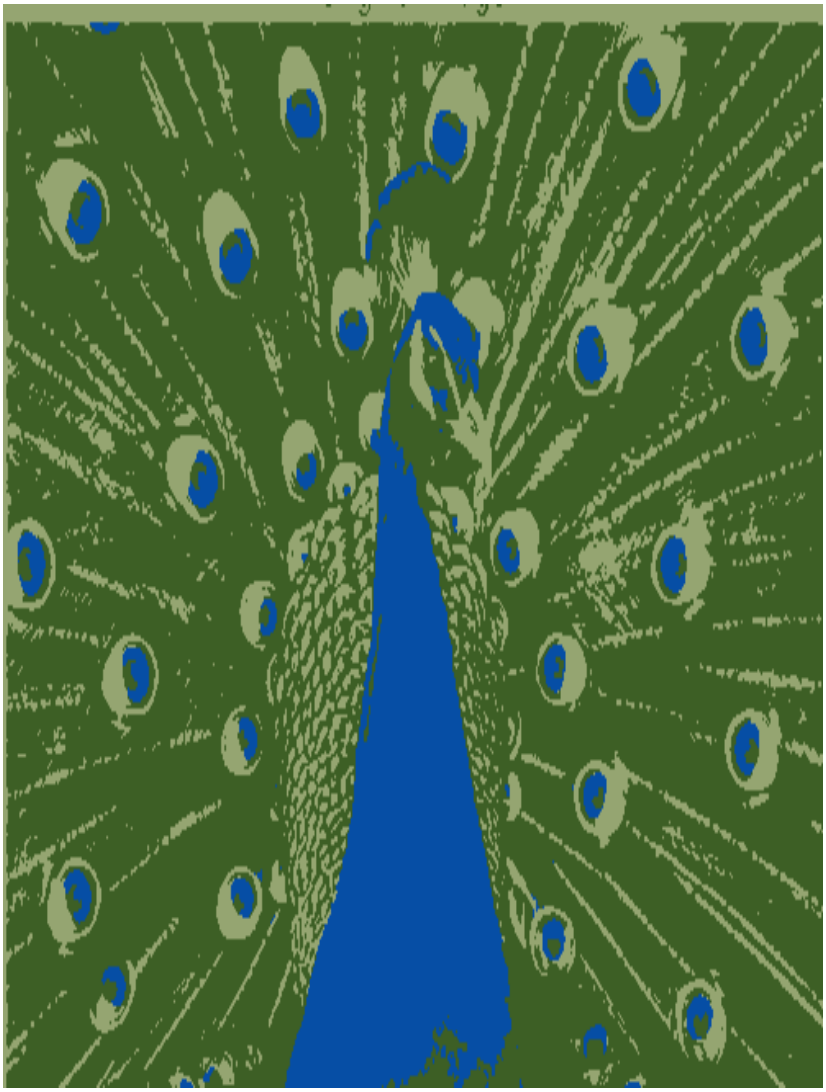
markers = cv2.watershed(
    cv2.cvtColor(watershed_img, cv2.COLOR_RGB2BGR),
    markers
)

watershed_img[markers == -1] = [255, 0, 0]

plt.figure(figsize=(8, 10))

plt.imshow(watershed_img)
plt.title("Watershed Segmentation")
plt.axis("off")

plt.show()
```



```
# Image ko pixels mein convert karo
pixel_data = img_rgb.reshape((-1, 3))

# Float mein convert
pixel_data = np.float32(pixel_data)

# Number of clusters
K = 3

# K-Means model
kmeans = KMeans(
    n_clusters=K,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixel_data)

# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)

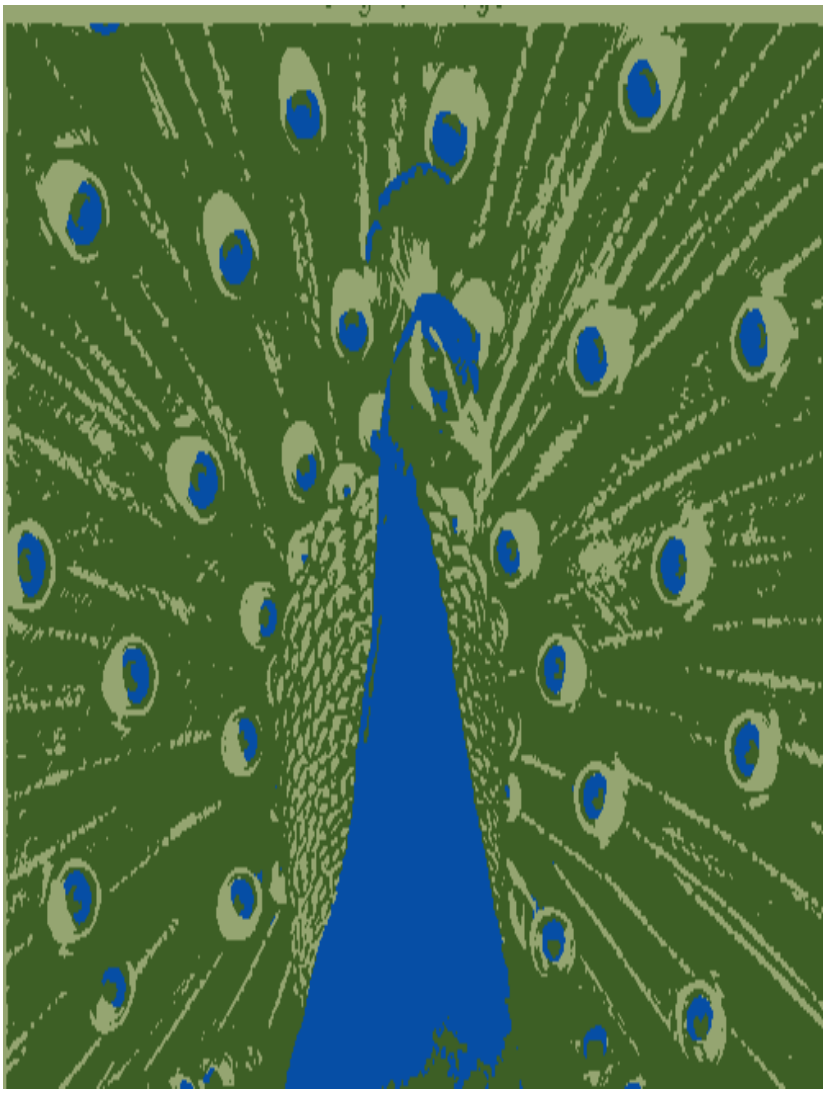
# Replace each pixel with its cluster center
segmented_data = centers[labels]

# Original shape mein reshape
segmented_image = segmented_data.reshape(img_rgb.shape)

plt.figure(figsize=(8, 10))

plt.imshow(segmented_image)
plt.title("K-Means Segmentation (K=3)")
plt.axis("off")
```

```
plt.show()
```



```
plt.figure(figsize=(16, 12))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")

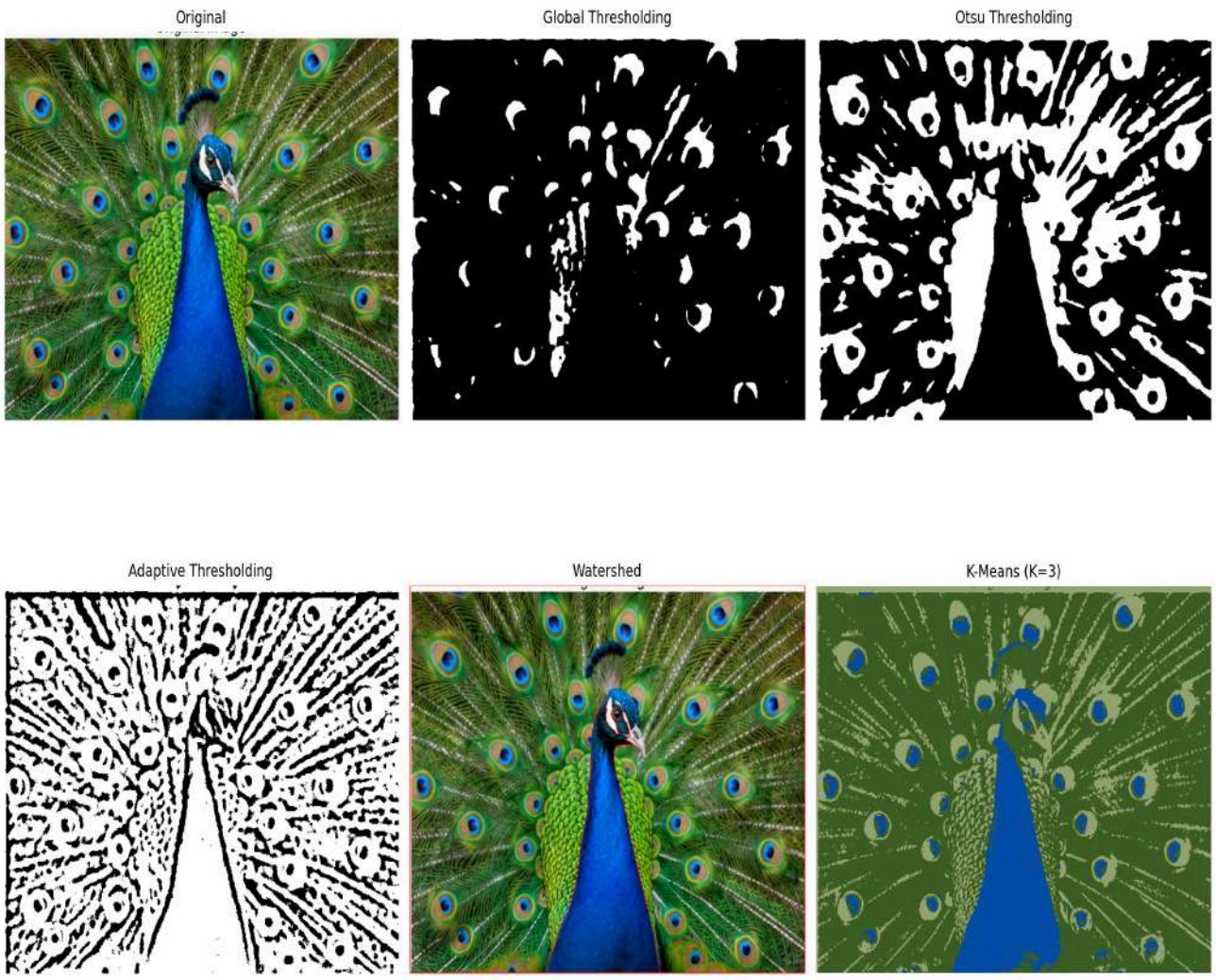
plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(watershed_img)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means (K=3)")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(6, 9))
plt.imshow(img_rgb)
plt.axis("off")
plt.tight_layout(pad=0)
plt.savefig(
    "original_image.png",
    dpi=300,
    bbox_inches="tight",
    pad_inches=0
)
plt.show()
```



```
# ============================================================
# EXPERIMENT NO. 6
# IMAGE SEGMENTATION TECHNIQUES
# Global Thresholding, Otsu, Adaptive, Watershed, K-Means
# ============================================================

# Install required library
!pip -q install scikit-learn

# -------------------- IMPORT LIBRARIES -----------------------
import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from google.colab import files
import zipfile
import os

print("Libraries loaded successfully!")
print("\nPlease upload your image...")

# -------------------- UPLOAD IMAGE ----------------------------
uploaded = files.upload()

filename = list(uploaded.keys())[0]

# Read image
img = cv2.imread(filename)

if img is None:
    raise ValueError("Image could not be loaded!")

# Convert BGR -> RGB
```

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# -------------------- GRAYSCALE ------------------------------
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -------------------- GAUSSIAN BLUR --------------------------
blur = cv2.GaussianBlur(gray, (15, 15), 0)

# ============================================================
# 1. GLOBAL THRESHOLDING
# ============================================================

_, global_thresh = cv2.threshold(
    blur,
    127,
    255,
    cv2.THRESH_BINARY
)

# ============================================================
# 2. OTSU'S THRESHOLDING
# ============================================================

otsu_value, otsu_thresh = cv2.threshold(
    blur,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# ============================================================
# 3. ADAPTIVE THRESHOLDING
# ============================================================

adaptive_thresh = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

# ============================================================
# 4. WATERSHED SEGMENTATION
# ============================================================

# Binary image
_, thresh_ws = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove noise
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    thresh_ws,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)
```

```
# Sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labelling
num_labels, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1

markers[unknown == 255] = 0

# Apply Watershed
watershed_bgr = img.copy()

markers = cv2.watershed(
    watershed_bgr,
    markers
)

# Convert result to RGB
watershed_rgb = cv2.cvtColor(
    watershed_bgr,
    cv2.COLOR_BGR2RGB
)

# Mark watershed boundaries in RED
watershed_rgb[markers == -1] = [255, 0, 0]

# ============================================================
# 5. K-MEANS COLOR SEGMENTATION
# ============================================================

# Resize image for faster processing
scale_width = 500
scale_height = int(
    img_rgb.shape[0] * scale_width / img_rgb.shape[1]
)

small_img = cv2.resize(
    img_rgb,
    (scale_width, scale_height)
)

# Convert pixels into data
pixel_data = small_img.reshape((-1, 3))
pixel_data = np.float32(pixel_data)

# Number of clusters
K = 3

# K-Means
kmeans = KMeans(
    n_clusters=K,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixel_data)

# Cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Replace pixels with cluster colors
segmented_data = centers[labels]
```

```
# Reshape back to image
kmeans_image = segmented_data.reshape(
    small_img.shape
)

# ============================================================
# DISPLAY ALL RESULTS
# ============================================================

print("\n========================================")
print("       IMAGE SEGMENTATION RESULTS")
print("========================================")

print("Original Image Size:", img.shape)
print("Otsu Automatically Selected Threshold:", otsu_value)
print("K-Means Number of Clusters:", K)

# Create figure
plt.figure(figsize=(18, 20))

# 1 Original
plt.subplot(4, 2, 1)
plt.imshow(img_rgb)
plt.title("1. Original Image", fontsize=14)
plt.axis("off")

# 2 Grayscale
plt.subplot(4, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("2. Grayscale Image", fontsize=14)
plt.axis("off")

# 3 Gaussian Blur
plt.subplot(4, 2, 3)
plt.imshow(blur, cmap="gray")
plt.title("3. Gaussian Blurred Image", fontsize=14)
plt.axis("off")

# 4 Global Threshold
plt.subplot(4, 2, 4)
plt.imshow(global_thresh, cmap="gray")
plt.title("4. Global Thresholding", fontsize=14)
plt.axis("off")

# 5 Otsu
plt.subplot(4, 2, 5)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("5. Otsu's Thresholding", fontsize=14)
plt.axis("off")

# 6 Adaptive
plt.subplot(4, 2, 6)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("6. Adaptive Thresholding", fontsize=14)
plt.axis("off")

# 7 Watershed
plt.subplot(4, 2, 7)
plt.imshow(watershed_rgb)
plt.title("7. Watershed Segmentation", fontsize=14)
plt.axis("off")

# 8 K-Means
plt.subplot(4, 2, 8)
plt.imshow(kmeans_image)
plt.title("8. K-Means Segmentation (K=3)", fontsize=14)
plt.axis("off")

plt.tight_layout()

# Save combined result
plt.savefig(
    "Experiment_6_All_Results.png",
    dpi=300,
    bbox_inches="tight"
)
```

```
plt.show()

# ============================================================
# SAVE INDIVIDUAL IMAGES
# ============================================================

def save_rgb_image(image, filename, title):
    plt.figure(figsize=(7, 10))
    plt.imshow(image)
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()
    plt.savefig(
        filename,
        dpi=300,
        bbox_inches="tight",
        pad_inches=0.1
    )
    plt.close()

def save_gray_image(image, filename, title):
    plt.figure(figsize=(7, 10))
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()
    plt.savefig(
        filename,
        dpi=300,
        bbox_inches="tight",
        pad_inches=0.1
    )
    plt.close()

save_rgb_image(
    img_rgb,
    "01_Original.png",
    "Original Image"
)

save_gray_image(
    gray,
    "02_Grayscale.png",
    "Grayscale Image"
)

save_gray_image(
    blur,
    "03_Gaussian_Blur.png",
    "Gaussian Blurred Image"
)

save_gray_image(
    global_thresh,
    "04_Global_Threshold.png",
    "Global Thresholding"
)

save_gray_image(
    otsu_thresh,
    "05_Otsu_Threshold.png",
    "Otsu's Thresholding"
)

save_gray_image(
    adaptive_thresh,
    "06_Adaptive_Threshold.png",
    "Adaptive Thresholding"
)

save_rgb_image(
    watershed_rgb,
    "07_Watershed.png",
    "Watershed Segmentation"
)

save_rgb_image(
```

```
    kmeans_image,
    "08_KMeans.png",
    "K-Means Segmentation"
)

# ============================================================
# CREATE ZIP FILE
# ============================================================

files_to_zip = [
    "Experiment_6_All_Results.png",
    "01_Original.png",
    "02_Grayscale.png",
    "03_Gaussian_Blur.png",
    "04_Global_Threshold.png",
    "05_Otsu_Threshold.png",
    "06_Adaptive_Threshold.png",
    "07_Watershed.png",
    "08_KMeans.png"
]

with zipfile.ZipFile(
    "Experiment_6_Results.zip",
    "w"
) as zipf:

    for file in files_to_zip:
        zipf.write(file)

print("\n========================================")
print("             EXPERIMENT COMPLETED!")
print("========================================")

print("\nGenerated files:")
for file in files_to_zip:
    print("✓", file)

print("\nOtsu Threshold =", otsu_value)
print("K-Means Clusters =", K)

print("\nAll images have been saved at 300 DPI.")
print("Downloading ZIP file...")

# Download everything
files.download("Experiment_6_Results.zip")
```

```
Libraries loaded successfully!

Please upload your image...
```

Choose Files WhatsApp I….10.30.jpeg

WhatsApp Image 2026-09-02 at 10.10.30.jpeg(image/jpeg) - 237929 bytes, last modified: 9/2/2026 - 100% done

```
Saving WhatsApp Image 2026-09-02 at 10.10.30.jpeg to WhatsApp Image 2026-09-02 at 10.10.30 (3).jpeg
```